Tuples

01 July 2025 20:29

A tuple is a built in data type in Python that is used to store multiple items in a single variable, similar to a list. But it is immutable, which means once created, its elements cannot be changed.

Features of Tuples

Feature	Description
Immutable	**Cannot be modified after creation.**
Ordered	**Elements have a defined order.**
Allow duplicates	**Same value can occur multiple times.**
Indexed	**Accessed via index like list.(t[0], t[1],…)**
Heterogeneous	**Can contain different data types**

Tuple Syntax:

```
My_tuple = (1, 2, 3)
```

- **Parentheses () are used.**
- **Elements are separated by commas(,).**

For e.g.,

```
my_tuple = ('apple', 'banana', 'grape', 'orange')
print(my_tuple)
print("First element:", my_tuple[0])
print("Last element: ",my_tuple[-1])
```

Output:

```
('apple', 'banana', 'grape', 'orange')
First element: apple
Last element:  orange
```

E.g., of mixed data type:

```
info = ('Apple', 25, 25.5, True)
print("Mixed data type in tuple:",info)
```

Output:

```
Mixed data type in tuple: ('Apple', 25, 25.5, True)
```

```
singleElement = (5)
print(type(singleElement))
singleElement = (5,)
print(type(singleElement))
```

Storing single element in the tuple.

Output:

```
<class 'int'>
<class 'tuple'>
```

```
<class 'int'>
<class 'tuple'>
```

```python
my_tuple = ('apple', 'banana', 'grape', 'orange')
my_tuple[1] = 'mango'
print(my_tuple)
```

Output:

```
    my_tuple[1] = 'mango'
    ~~~~~~~~^^^
TypeError: 'tuple' object does not support item assignment
```

Looping through a tuple

```python
my_tuple = ('apple', 'banana', 'grape', 'orange')
for fruit in my_tuple:
    print(fruit)
```

Output:

```
apple
banana
grape
orange
```

Tuple Operations

```python
a = (1, 2)
b = (3, 4)
print(a)
print(b)
c = a + b  #(1, 2, 3, 4) Concatenation of tuples
print(c)
d = a * 2  #(1, 2, 1, 2) Repeatition of a tuple
print(d)
print(len(a), len(b), len(c), len(d))
```

Output:

```
(1, 2)
(3, 4)
(1, 2, 3, 4)
(1, 2, 1, 2)
2 2 4 4
```

```python
person = ('Shiv', 42, 'USA')
name, age, country = person
print(name)
print(age)
print(country)
```

Output:

```
Shiv
42
USA
```

Why use tuples?

- **Faster than list (Better performance)**
- **Safer when you don't want data to be modified.**
- **Commonly used in:**
  - **Function returns (returning multiple values)**
  - **Dictionary, cheese (tuple can be hashable(applied to hash algorithms))**
  - **Coordinates, color codes, etc.**

Tuple functions and methods.

Function/Method	Description
len(t)	Number of elements.
t.index(x)	First index of element x.
t.count(x)	Number of occurrences.
min(t)/max(t)	Min or a Max value in tuple.

```python
a = (1, 2)
b = (3, 4)
print(a)
print(b)
c = a + b  #(1, 2, 3, 4) Concatenation of tuples
print(c)
d = a * 2  #(1, 2, 1, 2) Repeatition of a tuple
print(d)
idx =  d.index(2)
print('First index:', idx)
```

```python
nextIdx = d.index(2, idx+1)
print('Next index:',nextIdx)
nextIdx = d.index(2, nextIdx + 1)
print('Next index:',nextIdx)
```

Output:

```
(1, 2)
(3, 4)
(1, 2, 3, 4)
(1, 2, 1, 2)
First index: 1
Next index: 3
Traceback (most recent call last):
  File "c:\Users\bluej\Documents\Students\JiyaGaurDeensAcademy\Coding\TuplesInPython\tupleOne.py", line 61, in <modul
e>
    nextIdx = d.index(2, nextIdx + 1)
              ^^^^^^^^^^^^^^^^^^^^^^^
ValueError: tuple.index(x): x not in tuple
```

```python
a = (1, 2)
b = (3, 4)
print(a)
print(b)
c = a + b  #(1, 2, 3, 4) Concatenation of tuples
print(c)
d = a * 2  #(1, 2, 1, 2) Repeatition of a tuple
print(d)
idx =  d.index(2)
print('First index:', idx)
nextIdx = d.index(2, idx+1)
print('Next index:',nextIdx)
# nextIdx = d.index(2, nextIdx + 1)
# print('Next index:',nextIdx)
c = d.count(2)
print(f'{2} has occured {c} times')
```

Output:

```
(1, 2)
(3, 4)
(1, 2, 3, 4)
(1, 2, 1, 2)
First index: 1
Next index: 3
2 has occured 2 times
```